

1. Problem

Selon l'introduction du cours, quels sont les 3 "mots" qui résument la POO ?

- (a) Optimiser (améliorer les performances)
- (b) Compiler (exécuter le code)
- (c) Généraliser (réutiliser le code)
- (d) Protéger (éviter les erreurs)
- (e) Ranger (organiser le code)

Solution

Les 3 piliers sont : **Ranger** (organiser le code en regroupant données et comportements), **Protéger** (encapsuler pour éviter les erreurs), **Généraliser** (héritage et polymorphisme pour réutiliser).

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Correct.
- (e) Correct.

2. Problem

Analysez le code suivant :

```
Etudiant e1 = new Etudiant("Alice", 20);
Etudiant e2 = new Etudiant("Bob", 22);
Etudiant e3 = e1;
```

Combien d'objets sont créés en mémoire ?

- (a) 3 objets
- (b) 2 objets
- (c) 0 objet
- (d) 1 objet

Solution

2 objets sont créés en mémoire (avec les deux **new**).

e3 contient une **référence** vers le même objet que **e1**. Ce n'est pas une copie de l'objet, c'est la même adresse mémoire.

- (a) Incorrect.
- (b) Correct.
- (c) Incorrect.
- (d) Incorrect.

3. Problem

Soit le code suivant :

```
Etudiant e1 = new Etudiant("Alice", 20);
Etudiant e3 = e1;
e3.age = 25;
```

Quelle sera la valeur de `e1.age` après exécution ?

Solution

`e1.age` vaudra **25**. Puisque `e1` et `e3` pointent vers le même objet, modifier via `e3` modifie l'objet que `e1` référence aussi.

4. Problem

Que se passe-t-il quand on écrit `Personne jean = new Personne();` ?

- (a) Java copie une instance existante de `Personne`
- (b) Java crée une variable locale sans allouer de mémoire
- (c) Java alloue de la mémoire, appelle le constructeur, et retourne une référence
- (d) Java vérifie si un objet `Personne` existe déjà

Solution

Avec `new` : 1. Java alloue de la mémoire dans le tas (heap) 2. Le constructeur de la classe est appelé pour initialiser l'objet 3. Une référence vers cet espace mémoire est retournée et stockée dans la variable

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Incorrect.

5. Problem

Que se passe-t-il si vous définissez un constructeur `Personne(String nom)` sans définir de constructeur sans paramètres ?

- (a) `new Personne()` ne compilera plus
- (b) Java crée automatiquement un constructeur sans paramètres
- (c) Une exception sera levée à l'exécution
- (d) Le code fonctionne normalement

Solution

Dès que vous définissez un constructeur, le constructeur par défaut (sans paramètres) **disparaît** !

Si vous avez besoin des deux, vous devez définir explicitement le constructeur sans paramètres.

- (a) Correct.
- (b) Incorrect.
- (c) Incorrect.
- (d) Incorrect.

6. Problem

Que se passe-t-il à l'exécution du code suivant ?

```
Etudiant e = null;  
System.out.println(e.nom);
```

- (a) Erreur de compilation
- (b) Lance une `NullPointerException`
- (c) Affiche "null"

- (d) Affiche une chaîne vide

Solution

Le programme va **planter** avec une `NullPointerException`.

Message d'erreur : `Cannot invoke "... because "e" is null`

On ne peut pas appeler de méthode ou accéder à un attribut sur `null` car il n'y a pas d'objet.

- (a) Incorrect.
- (b) Correct.
- (c) Incorrect.
- (d) Incorrect.

7. Problem

Soit le code suivant :

```
String a = new String("Hello");
String b = new String("Hello");
String c = a;
```

Quelles expressions retournent `true` ?

- (a) `a.equals(b)`
- (b) `a.equals(c)`
- (c) `a == b`
- (d) `a == c`

Solution

- (a) `a == b` **false** (deux objets différents en mémoire)
 - (b) `a.equals(b)` **true** (même contenu "Hello")
 - (c) `a == c` **true** (même référence, même objet)
 - (d) `a.equals(c)` **true** (même contenu)
- (a) Correct.
 - (b) Correct.
 - (c) Incorrect.
 - (d) Correct.

8. Problem

Quel code est le plus sûr pour comparer une variable `nom` (potentiellement `null`) avec la chaîne "admin" ?

- (a) `nom.compareTo("admin") == 0`
- (b) `nom.equals("admin")`
- (c) `"admin".equals(nom)`
- (d) `nom == "admin"`

Solution

Mettre la constante à gauche : `"admin".equals(nom)`

Ainsi, même si `nom` est `null`, pas de crash (`equals` gère `null` en paramètre).

L'autre syntaxe `nom.equals("admin")` provoque une `NullPointerException` si `nom` est `null`.

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Incorrect.

9. **Problem**

Quels sont les modificateurs d'accès en Java ?

- (a) (default/package)
- (b) protected
- (c) final
- (d) public
- (e) static
- (f) private

Solution

Les 4 modificateurs d'accès sont : **private**, **public**, **protected** et (default/package-private). **static** et **final** ne sont **PAS** des modificateurs d'accès, mais des modificateurs de comportement.

- (a) Correct.
- (b) Correct.
- (c) Incorrect.
- (d) Correct.
- (e) Incorrect.
- (f) Correct.

10. **Problem**

Un attribut déclaré **private** est accessible depuis :

- (a) La classe et ses sous-classes
- (b) Partout dans le programme
- (c) La classe et le même package
- (d) La classe elle-même uniquement

Solution

Modificateur	Classe	Package	Sous-classe	Partout
private				

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

11. **Problem**

Un attribut déclaré **protected** est accessible depuis :

- (a) N'importe quelle classe de n'importe quel package

- (b) Les sous-classes (même dans d'autres packages)
- (c) Les classes du même package
- (d) La classe elle-même

Solution

protected donne accès à : - La classe elle-même - Les classes du même package - Les sous-classes (même dans d'autres packages)

C'est plus permissif que **private** mais moins que **public**.

- (a) Incorrect.
- (b) Correct.
- (c) Incorrect.
- (d) Correct.

12. Problem

À quoi sert l'annotation **@Override** ?

- (a) Elle rend la méthode publique
- (b) Elle améliore les performances de la méthode
- (c) Elle est obligatoire pour redéfinir une méthode
- (d) Elle permet au compilateur de vérifier qu'on redéfinit bien une méthode du parent

Solution

L'annotation **@Override** indique au compilateur qu'on redéfinit une méthode de la classe mère.

Avantages : - **Sécurité** : le compilateur vérifie que la méthode existe bien dans la classe mère (détecte les fautes de frappe) - **Lisibilité** : le code est plus clair pour le lecteur

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

13. Problem

Où doit se trouver l'appel à **super()** dans un constructeur de classe fille ?

- (a) N'importe où dans le constructeur
- (b) En dernière instruction du constructeur
- (c) En première instruction du constructeur
- (d) Après l'initialisation des attributs de la classe fille

Solution

super() doit être la **première instruction** du constructeur.

Le parent doit être construit avant l'enfant. Si vous oubliez **super()**, Java appelle automatiquement **super()** sans argument (ce qui échoue si le parent n'a pas de constructeur sans paramètres).

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Incorrect.

14. **Problem**

Soit `abstract class Forme { abstract double calculerSurface(); }`. Quel code est valide ?

- (a) Les trois sont valides
- (b) `Forme f = new Forme();`
- (c) `Forme f = new Forme() {};`
- (d) `Forme f = new Cercle(5.0);` (si `Cercle` extends `Forme`)

Solution

On ne peut **pas** instancier une classe abstraite avec `new Forme()`.

Mais on peut déclarer une variable de type `Forme` et l'initialiser avec une instance d'une classe concrète qui hérite de `Forme`.

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

15. **Problem**

En Java, une classe peut hériter de combien de classes ?

- (a) Une seule classe
- (b) Aucune, l'héritage n'existe pas en Java
- (c) Deux classes maximum
- (d) Autant de classes que souhaité

Solution

En Java, une classe ne peut hériter que d'**une seule classe** (héritage simple).

C'est pour éviter le "problème du diamant" qui existe en C++ avec l'héritage multiple.

Pour "simuler" l'héritage multiple, on utilise les **interfaces**.

- (a) Correct.
- (b) Incorrect.
- (c) Incorrect.
- (d) Incorrect.

16. **Problem**

Pour chaque affirmation, dites si elle est vraie :

- (a) "Un Maître a un Animal" Composition
- (b) "Une Voiture a un Moteur" Composition
- (c) "Un Rectangle a une Forme" Composition
- (d) "Un Chien est un Animal" Héritage
- (e) "Une Voiture est un Moteur" Héritage

Solution

- "Un Chien **est un** Animal" **Héritage**
- "Une Voiture **a un** Moteur" **Composition**
- "Une Voiture **est un** Moteur" Faux ! (n'a pas de sens)

- “Un Maître **a un** Animal” **Composition**
- “Un Rectangle **est une** Forme” (pas “a une”) **Héritage**

- (a) Correct.
- (b) Correct.
- (c) Incorrect.
- (d) Correct.
- (e) Incorrect.

17. Problem

Comment choisir entre héritage et composition ?

- (a) Utiliser l’héritage pour les attributs, la composition pour les méthodes
- (b) Toujours utiliser l’héritage, c’est plus simple
- (c) Si “X est un Y” Héritage ; Si “X a un Y” Composition
- (d) Toujours utiliser la composition, c’est plus moderne

Solution

La règle d’or : - Si on peut dire “X **est un** Y” (relation d’identité) **Héritage** - Si on peut dire “X **a un** Y” (relation de possession) **Composition**

Exemple : “Une Voiture est un Moteur” sonne faux, donc pas d’héritage. “Une Voiture a un Moteur” sonne juste, donc composition.

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Incorrect.

18. Problem

Quel mot-clé utilise-t-on pour implémenter une interface ?

- (a) inherits
- (b) implements
- (c) extends
- (d) uses

Solution

On utilise `implements` pour implémenter une interface :

```
class SiegeAuto implements ArticleLouable { ... }
```

On utilise `extends` pour hériter d’une classe.

- (a) Incorrect.
- (b) Correct.
- (c) Incorrect.
- (d) Incorrect.

19. Problem

Une classe peut-elle implémenter plusieurs interfaces ?

- (a) Maximum 2 interfaces

- (b) Non, une seule interface par classe
- (c) Oui, autant qu'elle veut
- (d) Oui, mais seulement si les interfaces n'ont pas de méthodes en commun

Solution

Oui, une classe peut implémenter **autant d'interfaces qu'elle veut** :

```
class Canard implements Volant, Nageur, Marcheur { ... }
```

C'est la solution de Java au problème de l'héritage multiple : on hérite d'une classe mais on implémente plusieurs interfaces.

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.
- (d) Incorrect.

20. Problem

Quel est l'ordre correct pour hériter d'une classe ET implémenter des interfaces ?

- (a) L'ordre n'a pas d'importance
- (b) class X implements I1, I2 extends Y
- (c) class X extends Y implements I1, I2

Solution

L'ordre est obligatoire : **extends AVANT implements**.

```
class SiegeAuto extends Accessoire implements ArticleLouable, Transportable {
    // ...
}
```

- (a) Incorrect.
- (b) Incorrect.
- (c) Correct.

21. Problem

Que retourne `bob instanceof Carnivore` si `Humain` implements `Omnivore` et `Omnivore extends Carnivore` ?

- (a) false
- (b) NullPointerException
- (c) true
- (d) Erreur de compilation

Solution

true !

Si `Omnivore extends Carnivore` et `Humain implements Omnivore`, alors un `Humain` est aussi un `Carnivore` (par transitivité).

`instanceof` vérifie toute la chaîne d'héritage/implémentation.

- (a) Incorrect.
- (b) Incorrect.

- (c) Correct.
- (d) Incorrect.

22. **Problem**

À quoi sert le Garbage Collector en Java ?

- (a) À compiler le code Java en bytecode
- (b) À vérifier les types à l'exécution
- (c) À optimiser les performances du code
- (d) À libérer automatiquement la mémoire des objets non référencés

Solution

Le **Garbage Collector** (GC) surveille la mémoire et supprime automatiquement les objets qui ne sont plus référencés par aucune variable.

Contrairement au C/C++, le développeur Java n'a pas à gérer manuellement la libération de la mémoire (pas de `free()` ou `delete`).

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

23. **Problem**

Dans une classe `Personnage`, comment valider qu'une compétence reste entre 3 et 15 ?

```
void augmenterCompeteance(Competeance c, int valeur) {  
    // ???  
}
```

- (a) Ignorer silencieusement les valeurs invalides
- (b) Retourner false si la valeur est invalide
- (c) Ne rien vérifier, l'utilisateur sait ce qu'il fait
- (d) Lever une `IllegalArgumentException` si la valeur est hors limites

Solution

La bonne pratique est de **lever une exception** pour signaler une erreur :

```
void augmenterCompeteance(Competeance c, int valeur) {  
    if (valeur < 3 || valeur > 15) {  
        throw new IllegalArgumentException(  
            "La compétence doit être entre 3 et 15");  
    }  
    competences.put(c, valeur);  
}
```

Cela garantit l'intégrité des données et informe clairement l'appelant de son erreur.

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

24. Problem

Soit une classe `abstract class Personnage` avec `abstract void attaqueSpeciale()`.
Que doit faire une classe `Mercenaire extends Personnage` ?

- (a) Implémenter obligatoirement la méthode `attaqueSpeciale()`
- (b) Rien, la méthode est héritée automatiquement
- (c) Appeler `super.attaqueSpeciale()`
- (d) Déclarer aussi la méthode comme `abstract`

Solution

Une classe qui hérite d'une classe abstraite **doit implémenter toutes les méthodes abstraites**, sinon elle doit elle-même être déclarée abstraite.

```
class Mercenaire extends Personnage {  
    @Override  
    void attaqueSpeciale() {  
        System.out.println("Tir de précision !");  
    }  
}
```

- (a) Correct.
- (b) Incorrect.
- (c) Incorrect.
- (d) Incorrect.

25. Problem

Quel est le rôle du bloc `catch` dans un `try-catch` ?

- (a) Lever une nouvelle exception
- (b) Empêcher les exceptions de se produire
- (c) Exécuter du code après le `try`, qu'il y ait erreur ou non
- (d) Intercepter et traiter une exception levée dans le bloc `try`

Solution

Le bloc `catch` **intercepte** les exceptions levées dans le bloc `try` et permet de les **traiter** (afficher un message, corriger l'erreur, etc.) au lieu que le programme plante.

```
try {  
    personnage.augmenterCompete(Compete.FORCE, 20);  
} catch (IllegalArgumentException e) {  
    System.out.println("Erreur : " + e.getMessage());  
}
```

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.

26. Problem

Pourquoi appeler `super()` dans le constructeur d'une classe fille ?

- (a) Pour créer une nouvelle instance de la classe mère

- (b) C'est optionnel et ne sert à rien
- (c) Pour copier les attributs de la classe mère
- (d) Pour initialiser la partie héritée de l'objet via le constructeur parent

Solution

`super()` appelle le constructeur de la classe mère pour **initialiser les attributs hérités**.

Le parent doit être construit **avant** l'enfant. Si vous oubliez `super()`, Java appelle automatiquement `super()` sans arguments (ce qui échoue si le parent n'a pas de constructeur sans paramètres).

```
class Mercenaire extends Personnage {  
    Mercenaire(String nom) {  
        super(nom); // Initialise les attributs de Personnage  
        augmenterCompetence(Competence.AGILITE, 5); // Bonus du Mercenaire  
    }  
}
```

- (a) Incorrect.
- (b) Incorrect.
- (c) Incorrect.
- (d) Correct.